

Chapter 12

Programming Languages for XML

We now turn to programming languages for semistructured data. All the widely used languages of this type apply to XML data, and might be used for semistructured data represented in other ways as well. In this chapter, we shall study three such languages. The first, XPath, is a simple language for describing sets of similar paths in a graph of semistructured data. XQuery is an extension of XPath that adopts something of the style of SQL. It allows iterations over sets, subqueries, and many other features that will be familiar from the study of SQL.

The third topic of this chapter is XSLT. This language was developed originally as a transformation language, capable of restructuring XML documents or turning them into printable (HTML) documents. However, its expressive power is actually quite similar to that of XQuery, and it is capable of producing XML results. Thus, it can serve as a query language for XML.

12.1 XPath

In this section, we introduce XPath. We begin with a discussion of the data model used in the most recent version of XPath, called XPath 2.0; this model is used in XQuery as well. This model plays a role analogous to the “bag of tuples of primitive-type components” that is used in the relational model as the value of a relation.

In later sections, we learn about XPath path expressions and their meaning. In general, these expressions allow us to move from elements of a document to some or all of their subelements. Using “axes,” we are also able to move within documents in a variety of ways, and to obtain the attributes of elements.

12.1.1 The XPath Data Model

As in the relational model, XPath assumes that all values — those it produces and those constructed in intermediate steps — have the same general “shape.” In the relational model, this “shape” is a bag of tuples. Tuples in a given bag all have the same number of components, and the components each have a primitive type, e.g., integer or string. In XPath, the analogous “shape” is *sequence of items*. An *item* is either:

1. A value of primitive type: integer, real, boolean, or string, for example.
2. A *node*. There are many kinds of nodes, but in our introduction, we shall only talk about three kinds:
 - (a) *Documents*. These are files containing an XML document, perhaps denoted by their local path name or a URL.
 - (b) *Elements*. These are XML elements, including their opening tags, their matched closing tag if there is one, and everything in between (i.e., below them in the tree of semistructured data that an XML document represents).
 - (c) *Attributes*. These are found inside opening tags, as we discussed in several places in Chapter 11.

The items in a sequence need not be all of the same type, although often they will be.

Example 12.1: Figure 12.1 is a sequence of four items. The first is the integer 10; the second is a string, and the third is a real. These are all items of primitive type.

```

10

"ten"

10.0

<Number base = "8">
  <Digit>1</Digit>
  <Digit>2</Digit>
</Number>

@val="10"
```

Figure 12.1: A sequence of five items

The fourth item is a node, and this node’s type is “element.” Notice that the element has tag `Number` with an attribute and two subelements with tag `Digit`. The last item is an attribute node. □

12.1.2 Document Nodes

While the documents to which XPath is applied can come from various sources, it is common to apply XPath to documents that are files. We can make a document node from a file by applying the function:

`doc(file name)`

The named file should be an XML document. We can name a file either by giving its local name or a URL if it is remote. Thus, examples of document nodes include:

```
doc("movies.xml")
doc("/usr/sally/data/movies.xml")
doc("infolab.stanford.edu/~hector/movies.xml")
```

Every XPath query refers to a document. In many cases, this document will be apparent from the context. For example, recall our discussion of XML-Schema keys in Section 11.4.6. We used XPath expressions to denote the selector and field(s) for a key. In that context, the document was “whatever document the schema definition is being applied to.”

12.1.3 Path Expressions

Typically, an XPath expression starts at the root of a document and gives a sequence of tags and slashes (/), say $/T_1/T_2/\cdots/T_n$. We evaluate this expression by starting with a sequence of items consisting of one node: the document. We then process each of $T_1, T_2, \dots$ in turn. To process T_i , consider the sequence of items that results from processing the previous tags, if any. Examine those items, in order, and find for each all its subelements whose tag is T_i . Those items are appended to the output sequence, in the order in which they appear in the document.

As a special case, the root tag T_1 for the document is considered a “subelement” of the document node. Thus, the expression $/T_1$ produces a sequence of one item, which is an element node consisting of the entire contents of the document. The difference may appear subtle; before we applied the expression $/T_1$, we had a document node representing the file, and after applying $/T_1$ to that node we have an element node representing the text in the file.

Example 12.2: Suppose our document is a file containing the XML text of Fig. 11.5, which we reproduce here as Fig. 12.2. The path expression $/StarMovieData$ produces the sequence of one element. This element has tag `<StarMovieData>`, of course, and it consists of everything in Fig. 12.2 except for line (1).

Now, consider the path expression

`/StarMovieData/Star/Name`

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <StarMovieData>
3)   <Star starID = "cf" starredIn = "sw">
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)     <Address>
10)      <Street>5 Locust Ln.</Street>
11)      <City>Malibu</City>
12)    </Address>
13)  </Star>
14)  <Star starID = "mh" starredIn = "sw">
15)    <Name>Mark Hamill</Name>
16)    <Street>456 Oak Rd.</Street>
17)    <City>Brentwood</City>
18)  </Star>
19)  <Movie movieID = "sw" starsOf = "cf", "mh">
20)    <Title>Star Wars</Title>
21)    <Year>1977</Year>
22)  </Movie>
23) </StarMovieData>

```

Figure 12.2: An XML document for applying path expressions

When we apply the `StarMovieData` tag to the sequence consisting of the document, we get the sequence consisting of the root element, as discussed above. Next, we apply to this sequence the tag `Star`. There are two subelements of the `StarMovieData` element that have tag `Star`. These are lines (3) through (12) for star Carrie Fisher and lines (14) through (18) for star Mark Hamill. Thus, the result of the path expression `/StarMovieData/Star` is the sequence of these two elements, in that order.

Finally, we apply to this sequence the tag `Name`. The first element has one `Name` subelement, at line (4). The second element also has one `Name` subelement, at line (15). Thus, the sequence

```

<Name>Carrie Fisher</Name>
<Name>Mark Hamill</Name>

```

is the result of applying the path expression `/StarMovieData/Star/Name` to the document of Fig. 12.2. □

12.1.4 Relative Path Expressions

In several contexts, we shall use XPath expressions that are *relative* to the current node or sequence of nodes.

- In Section 11.4.6 we talked about selector and field values that were really XPath expressions relative to a node or sequence of nodes for which we were defining a key.
- In Example 12.2 we talked about applying the XPath expression `Star` to the element consisting of the entire document, or the expression `Name` to a sequence of `Star` elements.

Relative expressions do not start with a slash. Each such expression must be applied in some context, which will be clear from its use. The similarity to the way files and directories are designated in a UNIX file system is not accidental.

12.1.5 Attributes in Path Expressions

Path expressions allow us to find all the elements within a document that are reached from the root along a particular kind of path (a sequence of tags). Sometimes, we want to find not these elements but rather the values of an attribute of those elements. If so, we can end the path expression by an attribute name preceded by an at-sign. That is, the path-expression form is $/T_1/T_2/\cdots/T_n/@A$.

The result of this expression is computed by first applying the path expression $/T_1/T_2/\cdots/T_n$ to get a sequence of elements. We then look at the opening tag of each element, in turn, to find an attribute A . If there is one, then the value of that attribute is appended to the sequence that forms the result.

Example 12.3: The path expression

`/StarMovieData/Star/@starID`

applied to the document of Fig. 12.2 finds the two `Star` elements and looks into their opening tags at lines (3) and (14) to find the values of their `starID` attributes. Both elements have this attribute, so the result sequence is "cf" "mh".
□

12.1.6 Axes

So far, we have only navigated through semistructured-data graphs in two ways: from a node to its children or to an attribute. XPath in fact provides a large number of *axes*, which are modes of navigation. Two of these axes are *child* (the default axis) and *attribute*, for which `@` is really a shorthand. At each step in a path expression, we can prefix a tag or attribute name by an axis name and a double-colon. For example,

```
/StarMovieData/Star/@starID
```

is really shorthand for:

```
/child::StarMovieData/child::Star/attribute::starID
```

Some of the other axes are parent, ancestor (really a proper ancestor), descendant (a proper descendant), next-sibling (any sibling to the right), previous-sibling (any sibling to the left), self, and descendant-or-self. The latter has a shorthand `//` and takes us from a sequence of elements to those elements and all their subelements, at any level of nesting.

Example 12.4: It might look hard to find, in the document of Fig. 12.2, all the cities where stars live. The problem is that Mark Hamill’s city is not nested within an `Address` element, so it is not reached along the same paths as Carrie Fisher’s cities. However, the path expression

```
//City
```

finds all the `City` subelements, at any level of nesting, and returns them in the order in which they appear in the document. That is, the result of this path expression is the sequence:

```
<City>Hollywood</City>
<City>Malibu</City>
<City>Brentwood</City>
```

which we obtain from lines (7), (11), and (17), respectively.

We could also use the `//` axis within the path expression. For example, should the document contain city information that wasn’t about stars (e.g., studios and their addresses), then we could restrict the paths that we consider to make sure that the city was a subelement of a `Star` element. For the given document, the path expression

```
/StarMovieData/Star//City
```

produces the same three `City` elements as a result. $\square$

Some of the other axes have shorthands as well. For example, `..` stands for parent, and `.` for self. We have already seen `@` for attribute and `/` for child.

12.1.7 Context of Expressions

In order to understand the meaning of an axis like parent, we need to explore further the view of data in XPath. Results of expressions are sequences of elements or primitive values. However, XPath expressions and their results do not exist in isolation; if they did, it would not make sense to ask for the “parent” of an element. Rather, there is normally a context in which the expression is

evaluated. In all our examples, there is a single document from which elements are extracted. If we think of an element in the result of some XPath expression as a reference to the element in the document, then it makes sense to apply axes like *parent*, *ancestor*, or *next-sibling* to the element in the sequence.

For example, we mentioned in Section 11.4.6 that keys in XML Schema are defined by a pair of XPath expressions. Key constraints apply to XML documents that obey the schema that includes the constraint. Each such document provides the context for the XPath expressions in the schema itself. Thus, it is permitted to use all the XPath axes in these expressions.

12.1.8 Wildcards

Instead of specifying a tag along every step of a path, we can use a *** to say “any tag.” Likewise, instead of specifying an attribute, *@** says “any attribute.”

Example 12.5: Consider the path expression

```
/StarMovieData/*/@*
```

applied to the document of Fig. 12.2. First, `/StarMovieData/*` takes us to every subelement of the root element. There are three: two stars and a movie. Thus, the result of this path expression is the sequence of elements in lines (3) through (13), (14) through (18), and (19) through (22).

However, the expression asks for the values of all the attributes of these elements. We therefore look for attributes among the outermost tags of each of these elements, and return their values in the order in which they appear in the document. Thus, the sequence

```
"cf" "sw" "mh" "sw" "sw" "cf" "mh"
```

is the result of the XPath query.

A subtle point is that the value of the `starsOf` attribute in line (19) is itself a sequence of items — strings `"cf"` and `"mh"`. XPath expands sequences that are part of other sequences, so all items are at the “top level,” as we showed above. That is, a sequence of items is not itself an item. □

12.1.9 Conditions in Path Expressions

As we evaluate a path expression, we can restrict ourselves to follow only a subset of the paths whose tags match the tags in the expression. To do so, we follow a tag by a condition, surrounded by square brackets. This condition can be anything that has a boolean value. Values can be compared by comparison operators such as `=` or `>=`. “Not equal” is represented as in C, by `!=`. A compound condition can be constructed by connecting comparisons with operators `or` or `and`.

The values compared can be path expressions, in which case we are comparing the sequences returned by the expressions. Comparisons have an implied

“there exists” sense; two sequences are related if any pair of items, one from each sequence, are related by the given comparison operator. An example should make this concept clear.

Example 12.6: The following path expression:

```
/StarMovieData/Star[//City = "Malibu"]/Name
```

returns the names of the movie stars who have at least one home in Malibu. To begin, the path expression `/StarMovieData/Star` returns a sequence of all the `Star` elements. For each of these elements, we need to evaluate the truth of the condition `//City = "Malibu"`. Here, `//City` is a path expression, but it, like any path expression in a condition, is evaluated relative to the element to which the condition is applied. That is, we interpret the expression assuming that the element were the entire document to which the path expression is applied.

We start with the element for Carrie Fisher, lines (3) through (13) of Fig. 12.2. The expression `//City` causes us to look for all subelements, nested zero or more levels deep, that have a `City` tag. There are two, at lines (7) and (11). The result of the path expression `//City` applied to the Carrie-Fisher element is thus the sequence:

```
<City>Hollywood</City>
<City>Malibu</City>
```

Each item in this sequence is compared with the value `"Malibu"`. An element whose type is a primitive value such as a string can be equated to that string, so the second item passes the test. As a result, the entire `Star` element of lines (3) through (13) satisfies the condition.

When we apply the condition to the second item, lines (14) through (18) for Mark Hamill, we find a `City` subelement, but its value does not match `"Malibu"` and this element fails the condition. Thus, only the Carrie-Fisher element is in the result of the path expression

```
/StarMovieData/Star[//City = "Malibu"]
```

We have still to finish the XPath query by applying to this sequence of one element the continuation of the path expression, `/Name`. At this stage, we search for a `Name` subelement of the Carrie-Fisher element and find it at line (4). Consequently, the query result is the sequence of one element, `<Name>Carrie Fisher</Name>`. □

Several other useful forms of condition are:

- An integer $[i]$ by itself is true only when applied the i th child of its parent.
- A tag $[T]$ by itself is true only for elements that have one or more subelements with tag T .

- Similarly, an attribute `[A]` by itself is true only for elements that have a value for the attribute `A`.

Example 12.7: Figure 12.3 is a variant of our running movie example, in which we have grouped all the movies with a common title as one `Movie` element, with subelements that have tag `Version`. The title is an attribute of the movie, and the year is an attribute of the version. Versions have `Star` subelements. Consider the XPath query, applied to this document:

```
/Movies/Movie/Version[1]/@year
```

It asks for the year in which the first version of each movie was made, and the result is the sequence "1933" "1984".

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <Movies>
3)   <Movie title = "King Kong">
4)     <Version year = "1933">
5)       <Star>Fay Wray</Star>
6)     </Version>
7)     <Version year = "1976">
8)       <Star>Jeff Bridges</Star>
9)       <Star>Jessica Lange</Star>
10)    </Version>
11)    <Version year = "2005" />
12)  </Movie>
13)  <Movie title = "Footloose">
14)    <Version year = "1984">
15)      <Star>Kevin Bacon</Star>
16)      <Star>John Lithgow</Star>
17)      <Star>Sarah Jessica Parker</Star>
18)    </Version>
19)  </Movie>
20) </Movies>

```

Figure 12.3: An XML document for applying path expressions

In more detail, there are four `Version` elements that match the path

```
/Movies/Movie/Version
```

These are at lines (4) through (6), (7) through (10), line (11), and lines (14) through (18), respectively. Of these, the first and last are the first children of their respective parents. The `year` attributes for these versions are 1933 and 1984, respectively. □

Example 12.8: The XPath query:

```
/Movies/Movie/Version[Star]
```

applied to the document of Fig. 12.3 returns three `Version` elements. The condition `[Star]` is interpreted as “has at least one `Star` subelement.” That condition is true for the `Version` elements of lines (4) through (6), (7) through (10), and (14) through (18); it is false for the element of line (11). □

```
<Products>
  <Maker name = "A">
    <PC model = "1001" price = "2114">
      <Speed>2.66</Speed>
      <RAM>1024</RAM>
      <HardDisk>250</HardDisk>
    </PC>
    <PC model = "1002" price = "995">
      <Speed>2.10</Speed>
      <RAM>512</RAM>
      <HardDisk>250</HardDisk>
    </PC>
    <Laptop model = "2004" price = "1150">
      <Speed>2.00</Speed>
      <RAM>512</RAM>
      <HardDisk>60</HardDisk>
      <Screen>13.3</Screen>
    </Laptop>
    <Laptop model = "2005" price = "2500">
      <Speed>2.16</Speed>
      <RAM>1024</RAM>
      <HardDisk>120</HardDisk>
      <Screen>17.0</Screen>
    </Laptop>
  </Maker>
```

Figure 12.4: XML document with product data — beginning

12.1.10 Exercises for Section 12.1

Exercise 12.1.1: Figures 12.4 and 12.5 are the beginning and end, respectively, of an XML document that contains some of the data from our running products exercise. Write the following XPath queries. What is the result of each?

```
<Maker name = "E">
  <PC model = "1011" price = "959">
    <Speed>1.86</Speed>
    <RAM>2048</RAM>
    <HardDisk>160</HardDisk>
  </PC>
  <PC model = "1012" price = "649">
    <Speed>2.80</Speed>
    <RAM>1024</RAM>
    <HardDisk>160</HardDisk>
  </PC>
  <Laptop model = "2001" price = "3673">
    <Speed>2.00</Speed>
    <RAM>2048</RAM>
    <HardDisk>240</HardDisk>
    <Screen>20.1</Screen>
  </Laptop>
  <Printer model = "3002" price = "239">
    <Color>false</Color>
    <Type>laser</Type>
  </Printer>
</Maker>
<Maker name = "H">
  <Printer model = "3006" price = "100">
    <Color>true</Color>
    <Type>ink-jet</Type>
  </Printer>
  <Printer model = "3007" price = "200">
    <Color>true</Color>
    <Type>laser</Type>
  </Printer>
</Maker>
</Products>
```

Figure 12.5: XML document with product data — end

- a) Find the amount of RAM on each PC.
- b) Find the price of each product of any kind.
- c) Find all the printer elements.
- ! d) Find the makers of laser printers.
- ! e) Find the makers of PC's and/or laptops.
- f) Find the model numbers of PC's with a hard disk of at least 200 gigabytes.
- !! g) Find the makers of at least two PC's.

Exercise 12.1.2: The document of Fig. 12.6 contains data similar to that used in our running battleships exercise. In this document, data about ships is nested within their class element, and information about battles appears inside each ship element. Write the following queries in XPath. What is the result of each?

- a) Find the names of all ships.
- b) Find all the `Class` elements for classes with a displacement larger than 35000.
- c) Find all the `Ship` elements for ships that were launched before 1917.
- d) Find the names of the ships that were sunk.
- ! e) Find the years in which ships having the same name as their class were launched.
- ! f) Find the names of all ships that were in battles.
- !! g) Find the `Ship` elements for all ships that fought in two or more battles.

12.2 XQuery

XQuery is an extension of XPath that has become a standard for high-level querying of databases containing data in XML form. This section will introduce some of the important capabilities of XQuery.

```

<Ships>
  <Class name = "Kongo" type = "bc" country = "Japan"
    numGuns = "8" bore = "14" displacement = "32000">
    <Ship name = "Kongo" launched = "1913" />
    <Ship name = "Hiei" launched = "1914" />
    <Ship name = "Kirishima" launched = "1915">
      <Battle outcome = "sunk">Guadalcanal</Battle>
    </Ship>
    <Ship name = "Haruna" launched = "1915" />
  </Class>
  <Class name = "North Carolina" type = "bb" country = "USA"
    numGuns = "9" bore = "16" displacement = "37000">
    <Ship name = "North Carolina" launched = "1941" />
    <Ship name = "Washington" launched = "1941">
      <Battle outcome = "ok">Guadalcanal</Battle>
    </Ship>
  </Class>
  <Class name = "Tennessee" type = "bb" country = "USA"
    numGuns = "12" bore = "14" displacement = "32000">
    <Ship name = "Tennessee" launched = "1920">
      <Battle outcome = "ok">Surigao Strait</Battle>
    </Ship>
    <Ship name = "California" launched = "1921">
      <Battle outcome = "ok">Surigao Strait</Battle>
    </Class>
  <Class name = "King George V" type = "bb"
    country = "Great Britain"
    numGuns = "10" bore = "14" displacement = "32000">
    <Ship name = "King George V" launched = "1940" />
    <Ship name = "Prince of Wales" launched = "1941">
      <Battle outcome = "damaged">Denmark Strait</Battle>
      <Battle outcome = "sunk">Malaya</Battle>
    </Ship>
    <Ship name = "Duke of York" launched = "1941">
      <Battle outcome = "ok">North Cape</Battle>
    </Ship>
    <Ship name = "Howe" launched = "1942" />
    <Ship name = "Anson" launched = "1942" />
  </Class>
</Ships>

```

Figure 12.6: XML document containing battleship data

Case Sensitivity of XQuery

XQuery is case sensitive. Thus, keywords such as `let` or `for` need to be written in lower case, just like keywords in C or Java.

12.2.1 XQuery Basics

XQuery uses the same model for values that we introduced for XPath in Section 12.1.1. That is, all values produced by XQuery expressions are sequences of items. Items are either primitive values or nodes of various types, including elements, attributes, and documents. Elements in a sequence are assumed to exist in the context of some document, as discussed in Section 12.1.7.

XQuery is a *functional language*, which implies that any XQuery expression can be used in any place that an expression is expected. This property is a very strong one. SQL, for example, allows subqueries in many places; but SQL does not permit, for example, any subquery to be any operand of any comparison in a where-clause. The functional property is a double-edged sword. It requires every operator of XQuery to make sense when applied to lists of more than one item, leading to some unexpected consequences.

To start, every XPath expression is an XQuery expression. There is, however, much more to XQuery, including FLWR (pronounced “flower”) expressions, which are in some sense analogous to SQL select-from-where expressions.

12.2.2 FLWR Expressions

Beyond XPath expressions, the most important form of XQuery expression involves clauses of four types, called *for*-, *let*-, *where*-, and *return*- (FLWR) clauses.¹ We shall introduce each type of clause in turn. However, we should be aware that there are options in the order and occurrences of these clauses.

1. The query begins with zero or more *for*- and *let*-clauses. There can be more than one of each kind, and they can be interlaced in any order, e.g., *for*, *for*, *let*, *for*, *let*.
2. Then comes an optional *where*-clause.
3. Finally, there is exactly one *return*-clause.

Example 12.9: Perhaps the simplest FLWR expression is:

```
return <Greeting>Hello World</Greeting>
```

It examines no data, and produces a value that is a simple XML element. □

¹There is also an *order-by* clause that we shall introduce in Section 12.2.10. For that reason, FLWR is a less common acronym for the principal form of XQuery query than is FLWOR.

Let Clauses

The simple form of a let-clause is:

let variable := expression

The intent of this clause is that the expression is evaluated and assigned to the variable for the remainder of the FLWR expression. Variables in XQuery must begin with a dollar-sign. Notice that the assignment symbol is `:=`, not an equal sign (which is used, as in XPath, in comparisons). More generally, a comma-separated list of assignments to variables can appear where we have shown one.

Example 12.10: One use of let-clauses is to assign a variable to refer to one of the documents whose data is used by the query. For example, if we want to query a document in file `stars.xml`, we can start our query with:

```
let $stars := doc("stars.xml")
```

In what follows, the value of `$stars` is a single doc node. It can be used in front of an XPath expression, and that expression will apply to the XML document contained in the file `stars.xml`. $\square$

For Clauses

The simple form of a for-clause is:

for variable in expression

The intent is that the expression is evaluated. The result of any expression is a sequence of items. The variable is assigned to each item, in turn, and what follows this for-clause in the query is executed once for each value of the variable. You will not be much deceived if you draw an analogy between an XQuery for-clause and a C for-statement. More generally, several variables may be set ranging over different sequences of items in one for-clause.

Example 12.11: We shall use the data suggested in Fig. 12.7 for a number of examples in this section. The data consists of two files, `stars.xml` in Fig. 12.7(a) and `movies.xml` in Fig. 12.7(b). Each of these files has data similar to what we used in Section 12.1, but the intent is that what is shown is just a small sample of the actual contents of these files.

Suppose we start a query:

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie
... something done with each Movie element
```

```

1) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
2) <Stars>
3)   <Star>
4)     <Name>Carrie Fisher</Name>
5)     <Address>
6)       <Street>123 Maple St.</Street>
7)       <City>Hollywood</City>
8)     </Address>
9)     <Address>
10)      <Street>5 Locust Ln.</Street>
11)      <City>Malibu</City>
12)    </Address>
13)  </Star>
    ... more stars
14) </Stars>

```

(a) Document stars.xml

```

15) <? xml version="1.0" encoding="utf-8" standalone="yes" ?>
16) <Movies>
17)   <Movie title = "King Kong">
18)     <Version year = "1933">
19)       <Star>Fay Wray</Star>
20)     </Version>
21)     <Version year = "1976">
22)       <Star>Jeff Bridges</Star>
23)       <Star>Jessica Lange</Star>
24)     </Version>
25)     <Version year = "2005" />
26)   </Movie>
27)   <Movie title = "Footloose">
28)     <Version year = "1984">
29)       <Star>Kevin Bacon</Star>
30)       <Star>John Lithgow</Star>
31)       <Star>Sarah Jessica Parker</Star>
32)     </Version>
33)   </Movie>
    ... more movies
34) </Movies>

```

(b) Document movies.xml

Figure 12.7: Data for XQuery examples

Boolean Values in XQuery

A comparison like `$x = 10` evaluates to true or false (strictly speaking, to one of the names `xs:true` or `xs:false` from the namespace for XML Schema). However, several other types of expressions can be interpreted as true or false, and so can serve as the value of a condition in a where-clause. The important coercions to remember are:

1. If the value is a sequence of items, then the empty sequence is interpreted as false and nonempty sequences as true.
2. Among numbers, 0 and NaN (“not a number,” in essence an infinite number) are false, and other numbers are true.
3. Among strings, the empty string is false and other strings are true.

Notice that `$movies/Movies/Movie` is an XPath expression that tells us to start with the document in file `movies.xml`, then go to the root `Movies` element, and then form the sequence of all `Movie` subelements. The body of the “for-loop” will be executed first with `$m` equal to the element of lines (17) through (26) of Fig. 12.7, then with `$m` equal to the element of lines (27) through (33), and then with each of the remaining `Movie` elements in the document. □

The Where Clause

The form of a where-clause is:

where condition

This clause is applied to an item, and the *condition*, which is an expression, evaluates to true or false. If the value is true, then the return-clause is applied to the current values of any variables in the query. Otherwise, nothing is produced for the current values of variables.

The Return Clause

The form of this clause is:

return expression

The result of a FLWR expression, like that of any expression in XQuery, is a sequence of items. The sequence of items produced by the expression in the return-clause is appended to the sequence of items produced so far. Note that although there is only one return-clause, this clause may be executed many

times inside “for-loops,” so the result of the query may be constructed in stages. We should not think of the return-clause as a “return-statement,” since it does not end processing of the query.

Example 12.12: Let us complete the query we started in Example 12.11 by asking for a list of all the star elements found among the versions of all movies. The query is:

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie
return $m/Version/Star
```

The first value of `$m` in the “for-loop” is the element of lines (17) through (26) of Fig. 12.7. From that `Movie` element, the XPath expression `/Version/Star` produces a sequence of the three `Star` elements at lines (19), (22), and (23). That sequence begins the result of the query.

```
<Star>Fay Wray</Star>
<Star>Jeff Bridges</Star>
<Star>Jessica Lange</Star>
<Star>Kevin Bacon</Star>
<Star>John Lithgow</Star>
<Star>Sarah Jessica Parker</Star>
...
```

Figure 12.8: Beginning of the result sequence for the query of Example 12.12

The next value of `$m` is the element of lines (27) through (33). Now, the result of the expression in the return-clause is the sequence of elements in lines (29), (30), and (31). Thus the beginning of the result sequence looks like that in Fig. 12.8. □

12.2.3 Replacement of Variables by Their Values

Let us consider a modification to the query of Example 12.12. Here, we want to produce not just a sequence of `<Star>` elements, but rather a sequence of `Movie` elements, each containing all the stars of movies with a given title, regardless of which version they starred in. The title will be an attribute of the `Movie` element.

Figure 12.9 shows an attempt that seems right, but in fact *is not correct*. The expression we return for each value of `$m` seems to be an opening `<Movie>` tag followed by the sequence of `Star` elements for that movie, and finally a closing `</Movie>` tag. The `<Movie>` tag has a `title` attribute that is a copy of the same attribute from the `Movie` element in file `movies.xml`. However, when we execute this program, what appears is:

Sequences of Sequences

We should remind the reader that sequences of items can have no internal structure. Thus, in Fig. 12.8, there is no separator between Jessica Lange and Kevin Bacon, or any grouping of the first three stars and the last three, even though these groups were produced by different executions of the return-clause.

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie
return <Movie title = $m/@title>$m/Version/Star</Movie>
```

Figure 12.9: Erroneous attempt to produce Movie elements

```
<Movie title = "$m/@title">$m/Version/Star</Movie>
<Movie title = "$m/@title">$m/Version/Star</Movie>
...
```

The problem is that, between tags, or as the value of an attribute, any text string is permissible. This return statement looks no different, to the XQuery processor, than the return of Example 12.9, where we really were producing text inside matching tags. In order to get text interpreted as XQuery expressions inside tags, we need to surround the text by curly braces.

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie
return <Movie title = {$m/@title}>{$m/Version/Star}</Movie>
```

Figure 12.10: Adding curly braces fixes the problem

The proper way to meet our goal is shown in Fig. 12.10. In this query, the expressions `$m/title` and `$m/Version/Star` inside the braces are properly interpreted as XPath expressions. The first is replaced by a text string, and the second is replaced by a sequence of `Star` elements, as intended.

Example 12.13: This example not only further illustrates the use of curly braces to force interpretation of expressions, but also emphasizes how any XQuery expression can be used wherever an expression of any kind is permitted. Our goal is to duplicate the result of Example 12.12, where we got a sequence of `Star` elements, but to make the entire sequence of stars be within a `Stars` element. We cannot use the trick of Fig. 12.10 with `Stars` in place of `Star`, because that would place many `Stars` tags around separate groups of stars.

```

let $starSeq := (
    let $movies := doc("movies.xml")
    for $m in $movies/Movies/Movie
    return $m/Version/Star
)
return <Stars>{$starSeq}</Stars>

```

Figure 12.11: Putting tags around a sequence

Figure 12.11 does the job. We assign the sequence of `Star` elements that results from the query of Example 12.12 to a local variable `$starSeq`. We then return that sequence, surrounded by tags, being careful to enclose the variable in braces so it is evaluated and not treated literally. $\square$

12.2.4 Joins in XQuery

We can join two or more documents in XQuery in much the same way as we join two or more relations in SQL. In each case we need variables, each of which ranges over elements of one of the documents or tuples of one of the relations, respectively. In SQL, we use a *from*-clause to introduce the needed tuple variables (which may just be the table name itself); in XQuery we use a *for*-clause.

However, we must be very careful how we do comparisons in a join. First, there is the matter of comparison operators such as `=` or `<` operating on sequences with the meaning of “there exist elements that compare” as discussed in Section 12.1.9. We shall take up this point again in Section 12.2.5. Additionally, equality of elements is by “element identity” (analogous to “object identity”). That is, an element is not equal to a different element, even if it looks the same, character-by-character. Fortunately, we usually do not want to compare elements, but really the primitive values such as strings and integers that appear as values of their attributes and subelements. The comparison operators work as expected on primitive values; `<` is “precedes in lexicographic order” for strings.

There is a built-in function `data(E)` that extracts the value of an element *E*. We can use this function to extract the text from an element that is a string with matching tags.

Example 12.14: Suppose we want to find the cities in which stars mentioned in the `movies.xml` file of Fig. 12.7(b) live. We need to consult the `stars.xml` file of Fig. 12.7(a) to get that information. Thus, we set up a variable ranging over the `Star` elements of `movies.xml` and another variable ranging over the `Star` elements of `stars.xml`. When the data in a `Star` element of `movies.xml` matches the data in the `Name` subelement of a `Star` element of `stars.xml`, then we have a match, and we extract the `City` element of the latter.

Figure 12.12 shows a solution. The `let`-clause introduces variables to stand for the two documents. As before, this shorthand is not necessary, and we could have used the document nodes themselves in the XPath expressions of the next two lines. The `for`-clause introduces a doubly nested loop. Variable `$s1` ranges over each `Star` element of `movies.xml` and `$s2` does the same for `stars.xml`.

```
let $movies := doc("movies.xml"),
    $stars  := doc("stars.xml")
for $s1 in $movies/Movies/Movie/Version/Star,
    $s2 in $stars/Stars/Star
where data($s1) = data($s2/Name)
return $s2/Address/City
```

Figure 12.12: Finding the cities of stars

The `where`-clause uses the built-in function `data` to extract the strings that are the values of the elements `$s1` and `$s2`. Finally, the `return`-clause produces a `City` element. $\square$

12.2.5 XQuery Comparison Operators

We shall now consider another puzzle where things don't quite work as expected. Our goal is to find the stars in `stars.xml` of Fig. 12.7(a) that live at 123 Maple St., Malibu. Our first attempt is in Fig. 12.13.

```
let $stars := doc("stars.xml")
for $s in $stars/Stars/Star
where $s/Address/Street = "123 Maple St." and
    $s/Address/City = "Malibu"
return $s/Name
```

Figure 12.13: An erroneous attempt to find who lives at 123 Maple St., Malibu

In the `where`-clause, we compare `Street` elements and `City` elements with strings, but that works as expected, because an element whose value is a string is coerced to that string, and the comparison will succeed when expected. The problem is seen when `$s` takes the `Star` element of lines (3) through (13) of Fig. 12.7 as its value. Then, XPath expression `$s/Address/Street` produces the sequence of two elements of lines (6) and (10) as its value. Since the `=` operator returns true if any pair of items, one from each side, equate, the value of the first condition is true; line (6), after coercion, is equal to the string "123 Maple St.". Similarly, the second condition compares the list of two `City` elements of lines (7) and (11) with the string "Malibu", and equality is found for line (11). As a result, the `Name` element for Carrie Fisher [line (4)] is returned.

But Carrie Fisher doesn't live at 123 Maple St., Malibu. She lives at 123 Maple St., Hollywood, and elsewhere in Malibu. The existential nature of comparisons has caused us to fail to notice that we were getting a street and city from different addresses.

XQuery provides a set of comparison operators that only compare sequences consisting of a single item, and fail if either operand is a sequence of more than one item. These operators are two-letter abbreviations for the comparisons: `eq`, `ne`, `lt`, `gt`, `le`, and `ge`. We could use `eq` in place of `=` to catch the case where we are actually comparing a string with several streets or cities. The revised query is shown in Fig. 12.14.

```
let $stars := doc("stars.xml")
for $s in $stars/Stars/Star
where $s/Address/Street eq "123 Maple St." and
      $s/Address/City eq "Malibu"
return $s/Name
```

Figure 12.14: A second erroneous attempt to find who lives at 123 Maple St., Malibu

This query does not allow the Carrie-Fisher element to pass the test of the where-clause, because the left sides of the `eq` operator are not single items, and therefore the comparison fails. Unfortunately, it will not report any star with two or more addresses, even if one of those addresses is 123 Maple St., Malibu. Writing a correct query is tricky, regardless of which version of the comparison operators we use, and we leave a correct query as an exercise.

12.2.6 Elimination of Duplicates

XQuery allows us to eliminate duplicates in sequences of any kind, by applying the built-in function `distinct-values`. There is a subtlety that must be noted, however. Strictly speaking, `distinct-values` applies to primitive types. It will strip the tags from an element that is a tagged text-string, but it won't put them back. Thus, the input to `distinct-values` can be a list of elements and the result a list of strings.

Example 12.15: Figure 12.11 gathered all the `Star` elements from all the movies and returned them as a sequence. However, a star that appeared in several movies would appear several times in the sequence. By applying `distinct-values` to the result of the subquery that becomes the value of variable `$starseq`, we can eliminate all but one copy of each `Star` element. The new query is shown in Fig. 12.15.

Notice, however, that what is produced is a list of the names of the stars surrounded by the `Stars` tags, as:

```
<Stars>"Fay Wray" "Jeff Bridges" ... </Stars>
```

```

let $starSeq := distinct-values(
  let $movies := doc("movies.xml")
  for $m in $movies/Movies/Movie
  return $m/Version/Star
)
return <Stars>{$starSeq}</Stars>

```

Figure 12.15: Eliminating duplicate stars

In comparison, the version in Fig. 12.11 produced

```

<Stars><Star>Fay Wray</Star> <Star>Jeff Bridges</Star> ...
</Stars>

```

but might produce duplicates. □

12.2.7 Quantification in XQuery

There are expressions that say, in effect, “for all” and “there exists.” Their forms, respectively, are:

```

every variable in expression1 satisfies expression2
some variable in expression1 satisfies expression2

```

Here, *expression1* produces a sequence of items, and the variable takes on each item, in turn, as its value. For each such value, *expression2* (which normally involves the variable) is evaluated, and should produce a boolean value.

In the “every” version, the result of the entire expression is false if some item produced by *expression1* makes *expression2* false; the result is true otherwise. In the “some” version, the result of the entire expression is true if some item produced by *expression1* makes *expression2* true; the result is false otherwise.

```

let $stars := doc("stars.xml")
for $s in $stars/Stars/Star
where every $c in $s/Address/City satisfies
  $c = "Hollywood"
return $s/Name

```

Figure 12.16: Finding the stars who only live in Hollywood

Example 12.16: Using the data in the file `stars.xml` of Fig. 12.7(a), we want to find those stars who live in Hollywood and nowhere else. That is, no matter how many addresses they have, they all have city Hollywood. Figure 12.16 shows how to write this query. Notice that `$s/Address/City` produces the

sequence of `City` elements of the star `$s`. The where-clause is thus satisfied if and only if every element on that list is `<City>Hollywood</City>`.

Incidentally, we could change the “every” to “some” and find the stars that have at least one home in Hollywood. However, it is rarely necessary to use the “some” version, since most tests in XQuery are existentially quantified anyway. For instance,

```
let $stars := doc("stars.xml")
for $s in $stars/Stars/Star
where $s/Address/City = "Hollywood"
return $s/Name
```

produces the stars with a home in Hollywood, without using a “some” expression. Recall our discussion in Section 12.2.5 of how a comparison such as `=`, with a sequence of more than one item on either or both sides, is true if we can match any items from the two sides. $\square$

12.2.8 Aggregations

XQuery provides built-in functions to compute the usual aggregations such as count, sum, or max. They take any sequence as argument; that is, they can be applied to the result of any XQuery expression.

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie
where count($m/Version) > 1
return $m
```

Figure 12.17: Finding the movies with multiple versions

Example 12.17: Let us examine the data in file `movies.xml` of Fig. 12.7(b) and produce those `Movie` elements that have more than one version. Figure 12.17 does the job. The XPath expression `$m/Version` produces the sequence of `Version` elements for the movie `$m`. The number of items in the sequence is counted. If that count exceeds 1, the where-clause is satisfied, and the movie element `$m` is appended to the result. $\square$

12.2.9 Branching in XQuery Expressions

There is an if-then-else expression in XQuery of the form

if (expression1) then expression2 else expression3

To evaluate this expression, first evaluate *expression1*; if it is true, evaluate *expression2*, which becomes the result of the whole expression. If *expression1* is false, the result of the whole expression is *expression3*.

This expression is not a statement — there are no statements in XQuery, only expressions. Thus, the analog in C is the `?:` expression, not the if-then-else statement. Like the expression in C, there is no way to omit the “else” part. However, we can use as *expression3* the empty sequence, which is denoted `()`. This choice makes the conditional expression produce the empty sequence when the test-condition is not satisfied.

Example 12.18: Our goal in this example is to produce each of the versions of *King Kong*, tagging the most recent version `Latest` and the earlier versions `Old`. In line (1), we set variable `$kk` to be the `Movie` element for *King Kong*. Notice that we have used an XPath condition in this line, to make sure that we produce only that one element. Of course, if there were several `Movie` elements that had the title *King Kong*, then all of them would be on the sequence of items that is the value of `$kk`, and the query would make no sense. However, we are assuming title is a key for movies in this structure, since we have explicitly grouped versions of movies with the same title.

```

1) let $kk :=
    doc("movies.xml")/Movies/Movie[@title = "King Kong"]
2) for $v in $kk/Version
3) return
4)   if ($v/@year = max($kk/Version/@year))
5)   then <Latest>{$v}</Latest>
6)   else <Old>{$v}</Old>

```

Figure 12.18: Tagging the versions of *King Kong*

Line (2) causes `$v` to iterate over all versions of *King Kong*. For each such version, we return one of two elements. To tell which, we evaluate the condition of line (4). On the right of the equal-sign is the maximum year of any of the *King-Kong* versions, and on the left is the year of the version `$v`. If they are equal, then `$v` is the latest version, and we produce the element of line (5). If not, then `$v` is an old version, and we produce the element of line (6). □

12.2.10 Ordering the Result of a Query

It is possible to sort the results as part of a FLWR query, if we add an order-clause before the return-clause. In fact, the query form we have been concentrating on here is usually called FLWOR (but still pronounced “flower”), to acknowledge the optional presence of an order-clause. The form of this clause is:

order list of expressions

The sort is based on the value of the first expression, ties are broken by the value of the second expression, and so on. The default order is ascending, but the keyword `descending` following an expression reverses the order.

What happens when an order is present is analogous to what happens in SQL. Just before we reach the stage in query processing where the output is assembled (the `SELECT` clause in SQL; the return-clause in XQuery), the result of previous clauses is assembled and sorted. In the case of SQL, the intermediate result is a set of bindings of tuples to the tuple variables that range over each of the relations in the `FROM` clause. Specifically, it is all those bindings that pass the test of the `WHERE` clause.

In XQuery, we should think of the intermediate result as a sequence of bindings of variables to values. The variables are those defined in the `for`- and `let`-clauses that precede the `order`-clause, and the sequence consists of all those bindings that pass the test of the `where`-clause. These bindings are each used to evaluate the expressions in the `order`-clause, and the values of those expressions govern the position of the binding in the order of all the bindings. Once we have the order of bindings, we use them, in turn, to evaluate the expression in the `return`-clause.

Example 12.19: Let us consider all versions of all movies, order them by year, and produce a sequence of `Movie` elements with the title and year as attributes. The data comes from file `movies.xml` in Fig. 12.7(b), as usual. The query is shown in Fig. 12.19.

```
let $movies := doc("movies.xml")
for $m in $movies/Movies/Movie,
    $v in $m/Version
order $v/@year
return <Movie title = "{$m/@title}" year = "{$v/@year}" />
```

Figure 12.19: Construct the sequence of title-year pairs, ordered by year

When we reach the `order`-clause, bindings provide values for the three variables `$movies`, `$m`, and `$v`. The value `doc("movies.xml")` is bound to `$movies` in every one of these bindings. However, the values of `$m` and `$v` vary; for each pair consisting of a movie and a version of that movie, there will be one binding for the two variables. For instance, the first such binding associates with `$m` the element in lines (17) through (26) of Fig. 12.7(b) and associates with `$v` the element of lines (18) through (20).

The bindings are sorted according to the value of attribute `year` in the element to which `$v` is bound. There may be many movies with the same year, and the ordering does not specify how these are to be ordered. As a result, all we know is that the movie-version pairs with a given year will appear together in some order, and the groups for each year will be in the ascending order of year. If we wanted to specify a total ordering of the bindings, we could, for example, add a second term to the list in the `order`-clause, such as:

```
order $v/@year, $m/@title
```

to break ties alphabetically by title.

After sorting the bindings, each binding is passed to the return-clause, in the order chosen. By substituting for the variables in the return-clause, we produce from each binding a single **Movie** element. $\square$

12.2.11 Exercises for Section 12.2

Exercise 12.2.1: Using the product data from Figs. 12.4 and 12.5, write the following in XQuery.

- a) Find the **Printer** elements with a price less than 100.
- b) Find the **Printer** elements with a price less than 100, and produce the sequence of these elements surrounded by a tag **<CheapPrinters>**.
- ! c) Find the names of the makers of both printers and laptops.
- ! d) Find the names of the makers that produce at least two PC's with a speed of 3.00 or more.
- ! e) Find the makers such that every PC they produce has a price no more than 1000.
- !! f) Produce a sequence of elements of the form

<Laptop><Model> x </Model><Maker> y </Maker></Laptop>

where x is the model number and y is the name of the maker of the laptop.

Exercise 12.2.2: Using the battleships data of Fig. 12.6, write the following in XQuery.

- a) Find the names of the classes that had at least 10 guns.
- b) Find the names of the ships that had at least 10 guns.
- c) Find the names of the ships that were sunk.
- d) Find the names of the classes with at least 3 ships.
- ! e) Find the names of the classes such that no ship of that class was in a battle.
- !! f) Find the names of the classes that had at least two ships launched in the same year.
- !! g) Produce a sequence of items of the form

<Battle name = x ><Ship name = y />...</Battle>

where x is the name of a battle and y the name of a ship in the battle. There may be more than one `Ship` element in the sequence.

- ! Exercise 12.2.3:** Solve the problem of Section 12.2.5; write a query that finds the star(s) living at a given address, even if they have several addresses, without finding stars that do not live at that address.
- ! Exercise 12.2.4:** Do there exist expressions E and F such that the expression `every $x in E satisfies F` is true, but `some $x in E satisfies F` is false? Either give an example or explain why it is impossible.

12.3 Extensible Stylesheet Language

XSLT (Extensible Stylesheet Language for Transformations) is a standard of the World-Wide-Web Consortium. Its original purpose was to allow XML documents to be transformed into HTML or similar forms that allowed the document to be viewed or printed. However, in practice, XSLT is another query language for XML. Like XPath or XQuery, we can use XSLT to extract data from documents or turn one document form into another form.

12.3.1 XSLT Basics

Like XML Schema, XSLT specifications are XML documents; these specifications are usually called *stylesheets*. The tags used in XSLT are found in a namespace, which is `http://www.w3.org/1999/XSL/Transform`. Thus, at the highest level, a stylesheet looks like Fig. 12.20.

```
<? xml version = "1.0" encoding = "utf-8" ?>
<xsl:stylesheet xmlns:xsl =
    "http://www.w3.org/1999/XSL/Transform">
    ...
</xsl:stylesheet>
```

Figure 12.20: The form of an XSLT stylesheet

12.3.2 Templates

A stylesheet will have one or more *templates*. To apply a stylesheet to an XML document, we go down the list of templates until we find one that matches the root. As processing proceeds, we often need to find matching templates for elements nested within the document. If so, we again search the list of templates for a match according to matching rules that we shall learn in this section. The simplest form of a template tag is:

```
<xsl:template match = "XPath expression">
```

The XPath expression, which can be either rooted (beginning with a slash) or relative, describes the elements of an XML document to which this template is applied. If the expression is rooted, then the template is applied to every element of the document that matches the path. Relative expressions are applied when a template *T* has within it a tag `<xsl:apply-templates>`. In that case, we look among the children of the elements to which *T* is applied. In that way, we can traverse an XML document's tree in a depth-first manner, performing complicated transformations on the document.

The simplest content of a template is text, typically HTML. When a template matches a document, the text inside that document is produced as output. Within the text can be calls to apply templates to the children and/or obtain values from the document itself, e.g., from attributes of the current element.

```

1)   <? xml version = "1.0" encoding = "utf-8" ?>
2)   <xsl:stylesheet xmlns:xsl =
3)       "http://www.w3.org/1999/XSL/Transform">
4)       <xsl:template match = "/">
5)           <HTML>
6)               <BODY>
7)                   <B>This is a document</b>
8)               </body>
9)           </html>
10)      </xsl:template>
11)  </xsl:stylesheet>
```

Figure 12.21: Printing output for any document

Example 12.20: In Fig. 12.21 is an exceedingly simple stylesheet. It applies to any document and produces the same HTML document, regardless of its input. This HTML document says “This is a document” in boldface.

Line (4) introduces the one template in the stylesheet. The value of the `match` attribute is `/`, which matches only the root. The body of the template, lines (5) through (9), is simple HTML. When these lines are produced as output, the resulting file can be treated as HTML and displayed by a browser or other HTML processor. □

12.3.3 Obtaining Values From XML Data

It is unusual that the document we produce does not depend in any way on the input to the transformation, as was the case in Example 12.20. The simplest way to extract data from the input is with the `value-of` tag. The form of this tag is:

```

<? xml version="1.0" encoding="utf-8" standalone="yes" ?>
<Movies>
  <Movie title = "King Kong">
    <Version year = "1933">
      <Star>Fay Wray</Star>
    </Version>
    <Version year = "1976">
      <Star>Jeff Bridges</Star>
      <Star>Jessica Lange</Star>
    </Version>
    <Version year = "2005" />
  </Movie>
  <Movie title = "Footloose">
    <Version year = "1984">
      <Star>Kevin Bacon</Star>
      <Star>John Lithgow</Star>
      <Star>Sarah Jessica Parker</Star>
    </Version>
  </Movie>
  ... more movies
</Movies>

```

Figure 12.22: The file movies.xml

```
<xsl:value-of select = "expression" />
```

The expression is an XPath expression that should produce a string as value. Other values, such as elements containing text, are coerced into strings in the obvious way.

Example 12.21: In Fig. 12.22 we reproduce the file `movies.xml` that was used in Section 12.2 as a running example. In this example of a stylesheet, we shall use `value-of` to obtain all the titles of movies and print them, one to a line. The stylesheet is shown in Fig. 12.23.

At line (4), we see that the template matches every `Movie` element, so we process them one at a time. Line (5) applies the `value-of` operation with an XPath expression `@title`. That is, we go to the `title` attribute of each `Movie` element and take the value of that attribute. This value is produced as output, and followed at line (6) by the HTML break tag, so the next movie title will be printed on the next line. □

12.3.4 Recursive Use of Templates

The most interesting and powerful transformations require recursive application of templates at various elements of the input. Having selected a template to

```
1)      <? xml version = "1.0" encoding = "utf-8" ?>
2)      <xsl:stylesheet xmlns:xsl =
3)          "http://www.w3.org/1999/XSL/Transform">
4)          <xsl:template match = "/Movies/Movie">
5)              <xsl:value-of select = "@title" />
6)              <BR/>
7)          </xsl:template>
8)      </xsl:stylesheet>
```

Figure 12.23: Printing the titles of movies

apply to the root of the input document, we can ask that a template be applied to each of its subelements, by using the `apply-templates` tag. If we want to apply a certain template to only some subset of the subelements, e.g., those with a certain tag, we can use a `select` expression, as:

```
<xsl:apply-templates select = "expression" />
```

When we encounter such a tag within a template, we find the set of matching subelements of the current element (the element to which the template is being applied). For each subelement, we find the first template that matches and apply it to the subelement.

Example 12.22: In this example, we shall use XSLT to transform an XML document into another XML document, rather than into an HTML document. Let us examine Fig. 12.24. There are four templates, and together they process movie data in the form of Fig. 12.22. The first template, lines (4) through (8), matches the root. It says to output the text `<Movies>` and then apply templates to the children of the root element. We could have specified that templates were to be applied only to children that are tagged `<Movie>`, but since we expect no other tags among the children, we did not specify:

```
6)      <xsl:apply-templates select = "Movie" />
```

Notice that after applying templates to the `<Movie>` children (which will result in the printing of many elements), we close the `<Movies>` element in the output with the appropriate closing tag at line (7). Also observe that we can tell the difference between tags that are output text, such as lines (5) and (7), from tags that are XSLT, because all XSLT tags must be from the `xsl` namespace.

Now, let us see what applying templates to the `<Movie>` elements does. The first (and only) template that matches these elements is the second, at lines (9) through (15). This template begins by outputting the text `<Movie title = "` at line (10). Then, line (11) obtains the title of the movie and emits it to the output. Line (12) finishes the quoted attribute value and the `<Movie>` tag in the output. Line (13) applies templates to all the children of the movie, which should be versions. Finally, line (14) emits the matching `</Movie>` ending tag.

```
1)    <? xml version = "1.0" encoding = "utf-8" ?>
2)    <xsl:stylesheet xmlns:xsl =
3)        "http://www.w3.org/1999/XSL/Transform">

4)        <xsl:template match = "/Movies">
5)            <Movies>
6)                <xsl:apply-templates />
7)            </Movies>
8)        </xsl:template>

9)        <xsl:template match = "Movie">
10)            <Movie title = "
11)                <xsl:value-of select = "@title" />
12)            ">
13)            <xsl:apply-templates />
14)        </Movie>
15)    </xsl:template>

16)    <xsl:template match = "Version">
17)        <xsl:apply-templates />
18)    </xsl:template>

19)    <xsl:template match = "Star">
20)        <Star name = "
21)            <xsl:value-of select = "." />
22)        " />
23)    </xsl:template>

24) </xsl:stylesheet>
```

Figure 12.24: Transforming the `movies.xml` file

When line (13) calls for templates to be applied to all the versions of a movie, the only matching template is that of lines (16) through (18), which does nothing but apply templates to the children of the version, which should be `<Star>` elements. Thus, what gets generated between each opening `<Movie>` tag and its matched closing tag is determined by the last template of lines (19) through (23). This template is applied to each `<Star>` element.

Star elements from the input are transformed in the output. Instead of the star's name being text, as it is in Fig. 12.22, the template starting at line (19) produces a `<Star>` element with the name as an attribute. Line (21) says to select the `<Star>` element itself (the dot represents the "self" axis as an XPath expression) as a value for the output. However, all output is text, so the tags of the element are not part of the output. That result is exactly what we want,

since the value of the attribute `name` should be a string, not an element. The empty `<Star>` element is completed on line (22). For instance, given the input of Fig. 12.22, the output would be as shown in Fig. 12.25. $\square$

```

<Movies>
  <Movie title = "King Kong">
    <Star name = "Fay Wray" />
    <Star name = "Jeff Bridges" />
    <Star name = "Jessica Lange" />
  </Movie>
  <Movie title = "Footloose">
    <Star name = "Kevin Bacon" />
    <Star name = "John Lithgow" />
    <Star name = "Sarah Jessica Parker" />
  </Movie>
  ... more movies
</Movies>

```

Figure 12.25: Output of the transform of Fig. 12.24

12.3.5 Iteration in XSLT

We can put a loop within a template that gives us freedom over the order in which we visit certain subelements of the element to which the template is being applied. The `for-each` tag creates the loop, with a form:

```
<xsl:for-each select = "expression">
```

The expression is an XPath expression whose value is a sequence of items. Whatever is between the opening `<for-each>` tag and its matched closing tag is executed for each item, in turn.

Example 12.23: In Fig. 12.26 is a copy of our document `stars.xml`; we wish to transform it to an HTML list of all the names of stars followed by an HTML list of all the cities in which stars live. Figure 12.27 has a template that does the job.

There is one template, which matches the root. The first thing that happens is at line (5), where the HTML tag `<OL>` is emitted to start an ordered list. Then, line (6) starts a loop, which iterates over each `<Star>` subelement. At lines (7) through (9), a list item with the name of that star is emitted. Line (11) ends the list of names and begins a list of cities. The second loop, lines (12) through (16), runs through each `<Address>` element and emits a list item for the city. Line (17) closes the second list. $\square$

```

<? xml version="1.0" encoding="utf-8" standalone="yes" ?>
<Stars>
  <Star>
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
  </Star>
  ... more stars
</Stars>

```

Figure 12.26: Document stars.xml

```

1)  <? xml version = "1.0" encoding = "utf-8" ?>
2)  <xsl:stylesheet xmlns:xsl =
3)    "http://www.w3.org/1999/XSL/Transform">
4)    <xsl:template match = "/">
5)      <OL>
6)        <xsl:for-each select = "Stars/Star" />
7)          <LI>
8)            <xsl:value-of select = "Name">
9)              </li>
10)          </xsl:for-each>
11)          </ol><P/><OL>
12)          <xsl:for-each select = "Stars/Star/Address" />
13)            <LI>
14)              <xsl:value-of select = "City">
15)                </li>
16)            </xsl:for-each>
17)          </ol>
18)        </xsl:template>
19)      </xsl:stylesheet>

```

Figure 12.27: Printing names and cities of stars

12.3.6 Conditionals in XSLT

We can introduce branching into our templates by using an if tag. The form of this tag is:

```
<xsl:if test = "boolean expression">
```

Whatever appears between this tag and its matched closing tag is executed if and only if the boolean expression is true. There is no else-clause, but we can follow this expression by another if that has the opposite test condition should we wish.

```

1)    <? xml version = "1.0" encoding = "utf-8" ?>
2)    <xsl:stylesheet xmlns:xsl =
3)        "http://www.w3.org/1999/XSL/Transform">
4)        <xsl:template match = "/">
5)            <TABLE border = "5"><TR><TH>Stars</th></tr>
6)            <xsl:for-each select = "Stars/Star" />
7)                <xsl:if test = "Address/City = 'Hollywood'">
8)                    <TR><TD>
9)                        <xsl:value-of select = "Name" />
10)                   </td></tr>
11)                </xsl:if>
12)            </xsl:for-each>
13)        </table>
14)    </xsl:template>
15) </xsl:stylesheet>
```

Figure 12.28: Finding the names of the stars who live in Hollywood

Example 12.24: Figure 12.28 is a stylesheet that prints a one-column table, with header “Stars.” There is one template, which matches the root. The first thing this template does is print the header row at line (5). The for-each loop of lines (6) through (12) iterates over each star. The conditional of line (7) tests whether the star has at least one home in Hollywood. Remember that the equal-sign represents a comparison is true if any item on the left equals any item on the right. That is what we want, since we asked whether any of the homes a star has is in Hollywood. Lines (8) through (10) print a row of the table. □

12.3.7 Exercises for Section 12.3

Exercise 12.3.1: Suppose our input XML document has the form of the product data of Figs. 12.4 and 12.5. Write XSLT stylesheets to produce each of the following documents.

- a) An HTML file consisting of a header “Manufacturers” followed by an enumerated list of the names of all the makers of products listed in the input.
- b) An HTML file consisting of a table with headers “Model” and “Price,” with a row for each PC. That row should have the proper model and price for the PC.
- ! c) An HTML file consisting of a table whose headers are “Model,” “Price,” “Speed,” and “Ram” for all Laptops, followed by another table with the same headers for PC’s.
- d) An XML file with root tag `<PCs>` and subelements having tag `<PC>`. This tag has attributes `model`, `price`, `speed`, and `ram`. In the output, there should be one `<PC>` element for each `<PC>` element of the input file, and the values of the attributes should be taken from the corresponding input element.
- !! e) An XML file with root tag `<Products>` whose subelements are `<Product>` elements. Each `<Product>` element has attributes `type`, `maker`, `model`, and `price`, where the type is one of “PC”, “Laptop”, or “Printer”. There should be one `<Product>` element in the output for every PC, laptop, and printer in the input file, and the output values should be chosen appropriately from the input data.
- ! f) Repeat part (b), but make the output file a Latex file.

Exercise 12.3.2: Suppose our input XML document has the form of the product data of Fig. 12.6. Write XSLT stylesheets to produce each of the following documents.

- a) An HTML file with a header for each class. Under each header is a table with column-headers “Name” and “Launched” with the appropriate entry for each ship of the class.
- b) An HTML file with root tag `<Losers>` and subelements `<Ship>`, each of whose values is the name of one of the ships that were sunk.
- ! c) An XML file with root tag `<Ships>` and subelements `<Ship>` for each ship. These elements each should have attributes `name`, `class`, `country` and `numGuns` with the appropriate values taken from the input file.
- ! d) Repeat (c), but only list those ships that were in at least one battle.
- e) An XML file identical to the input, except that `<Battle>` elements should be empty, with the outcome and name of the battle as two attributes.

12.4 Summary of Chapter 12

- ◆ *XPath*: This language is a simple way to express many queries about XML data. You describe paths from the root of the document by sequences of tags. The path may end at an attribute rather than an element.
- ◆ *The XPath Data Model*: All XPath values are sequences of items. An item is either a primitive value or an element. An element is an opening XML tag, its matched closing tag, and everything in between.
- ◆ *Axes*: Instead of proceeding down the tree in a path, one can follow another axis, including jumps to any descendant, a parent, or a sibling.
- ◆ *XPath Conditions*: Any step in a path can be constrained by a condition, which is a boolean-valued expression. This expression appears in square brackets.
- ◆ *XQuery*: This language is a more advanced form of query language for XML documents. It uses the same data model as XPath. XQuery is a functional language.
- ◆ *FLWR Expressions*: Many queries in XQuery consist of let-, for-, where- and return-clauses. “Let” introduces temporary definitions of variables; “for” creates loops; “where” supplies conditions to be tested, and “return” defines the result of the query.
- ◆ *Comparison Operators in XQuery and XPath*: The conventional comparison operators such as < apply to sequences of items, and have a “there-exists” meaning. They are true if the stated relation holds between any pair of items, one from each of the lists. To be assured that single items are being compared, we can use letter codes for the operators, such as lt for “less than.”
- ◆ *Other XQuery Expressions*: XQuery has many operations that resemble those in SQL. These operators include existential and universal quantification, aggregation, duplicate-elimination, and sorting of results.
- ◆ *XSLT*: This language is designed for transformations of XML documents, although it also can be used as a query language. A “program” in this language has the form of an XML document, with a special namespace that allows us to use tags to describe a transformation.
- ◆ *Templates*: The heart of XSLT is a template, which matches certain elements of the input document. The template describes output text, and can extract values from the input document for inclusion in the output. A template can also call for templates to be applied recursively to the children of an element.

- ◆ *XSLT Programming Constructs*: A template can also include XSLT constructs that behave like an iterative programming language. These constructs include for-loops and if-statements.

12.5 References for Chapter 12

The World-Wide-Web Consortium site for the definition of XPath is [2]. The site for XQuery is [3], and for XSLT it is [4].

[1] is an introduction to the XQuery language. There are tutorials for XPath, XQuery, and XSLT at [5].

1. D. D. Chamberlin, “XQuery: an XML Query Language,” *IBM Systems Journal* 41:4 (2002), pp. 597–615. See also www.research.ibm.com/journal/sj/414/chamberlin.pdf
2. World-Wide-Web Consortium <http://www.w3.org/TR/xpath>
3. World-Wide-Web Consortium <http://www.w3.org/TR/xquery>
4. World-Wide-Web Consortium <http://www.w3.org/TR/xslt>
5. W3 Schools, <http://www.w3schools.com>

Part IV

Database System Implementation

